

`-c <NumOfCPUs>` to allocate more than the default CPUs (1 for current version), `-m <AmountOfMemory>` to allocate memory other than the default memory (1G for current version), and `-d <DiskSpace>` to allocate disk space other than the default disk (5G for current version), respectively. You can also provide a timeout (default is 5 minutes for the current version) using `--timeout <TimeoutInSecs>` to limit the maximum time it takes to build the VM instance. For example, the command `multipass launch -n mycloudvm -c 2 -m 2G -d 10G --timeout 600` builds a local VM with the default LTS release version of Ubuntu (22.04 as of writing) named mycloudvm. It has 2 CPUs, 2GB memory and a 10GB disk with the launch operation timeout of 10 minutes.

You can use the command `multipass list` to dump the complete information about all the launched instances. The command `multipass info --all` also dumps information about all the instances, albeit with some extra runtime details. This information can be dumped in a table other than the default format `table` by appending `--format <json|csv|yaml>` to the `list` or `info` command. Once your instance is up, use the command `multipass shell <VMName>` to log in. Now your local cloud VM instance is ready to be used as per your need.

Please note that an instance name `primary` with default resources is created (if not done already) in case you don't provide the VM name to the shell command. The primary instance auto mounts your home directory into it, but you could unmount that using the `multipass unmount primary` command. If you want to set the name of the primary instance other than the default, then use the command `multipass set client.primary-name=<CustomPrimaryName>`. Similarly, the command `multipass start` launches a primary instance if not created already.

You can execute any shell command on a running instance through `multipass exec`. Logically, you could combine `transfer` and `exec` to copy and execute your configuration scripts to create some automation around the instances. The Multipass commands `stop`, `start`, `restart`, `suspend`, and `delete` are self-explanatory, and perform those operations on the named instances (`--all` for all the instances). The Multipass command `recover` is for recovering the deleted instances and `purge` cleans everything for these deleted instances. Multipass commands have some other options too, and you can dump detailed information about any command using `multipass help <Command>`.

Advanced tooling with Multipass

Multipass helps use cloud-init to fully configure local VMs as is done in various public clouds. This means you can prototype your public cloud launches locally, and for free. Isn't that a great ability, which is not found in any other similar tool? The cloud-init is used to initialise the VM instances without using any external tool for configuration, to build the fully configured machines out-of-the-box. For example, I'm using this functionality to build fully configured single instances running many standard cloud components on my laptop. It enables me and others to quickly experiment/verify/clean up combinations of various stacks, sometimes a few dozen times a day, before the final public cloud push.

Let's quickly develop the cloud-init configuration using the cloud-config data format, showing the various required sections. You can download and check the script from the cloud-config examples link given under References at the end of the article. We'll use it with Multipass to bring up a fully configured instance running Apache Kafka without the need for Zookeeper due to Kraft, and its web UI, out-of-the-box. Now let's dissect this cloud-init configuration

section by section. The `users` section is configuring a user Ubuntu with sudo privileges, ready to use your desired ssh keypair for the ssh login. Please note that the public key is a place holder and you need to replace it with the public key content of the key pair. If you don't have a ssh key pair, then you generate one by using the command `ssh-keygen` and following its question prompts. The `package_update` section is about resynchronising the package index files from their sources. The `package_upgrade` section is about installing the newest versions of all packages currently installed on the system. The `packages` section takes a list of the packages that need to get installed on the system. Our example configuration is installing the `avahi-daemon` so that we can resolve `<hostname>.local` to the instance address.

The section `write_files` generates the files listed by taking the content, path, owner and permission for each file. In our example configuration, we are generating a Docker Compose manifest to bring up Cassandra and its web UI stack. The `runcmd` section takes a list of the shell commands to run in order to complete the configuration of the instance. Finally, the `output` section generates different logs, capturing `stdout` and `stderr` during various stages of the cloud-init, to debug in case the instance is not configured as per the cloud-config data provided. The cloud-init cloud-config provides more functionality through many other sections. Please refer to the cloud config examples link in the References section at the end of the article to see these in action.

Execute the command `multipass launch -m 2G n kafka-kraft --cloud-init ./cloud-config-kafka-kraft.yaml` to bring up a fully configured cloud VM instance locally. You can try to access the Kafka cluster web UI by typing `kafka-kraft.local:8080` in your browser once the instance is up. Some of the web UI screenshots from my laptop are shown in Figures 5 and 6.